

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования

**«Пермский национальный исследовательский политехнический
университет»**

Электротехнический факультет

Выпускающая кафедра: Информационные технологии и
автоматизированные системы (ИТАС)

Направление подготовки: 09.03.04 Программная инженерия

ОТЧЕТ

Лабораторная работа

«Быстрые сортировки»

По дисциплине «Основы алгоритмизации и программирования»

Выполнил: студент группы РИС-25-2Б

Ильинов Фёдор Михайлович

Принял: Доц. Полякова О. А.

Пермь 2026

ПОСТАНОВКА ЗАДАЧИ

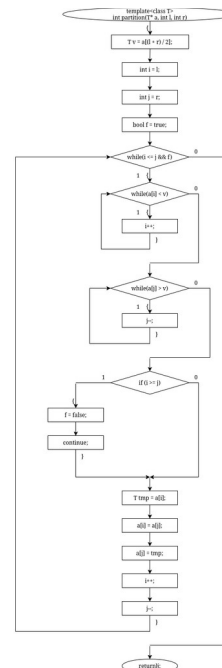
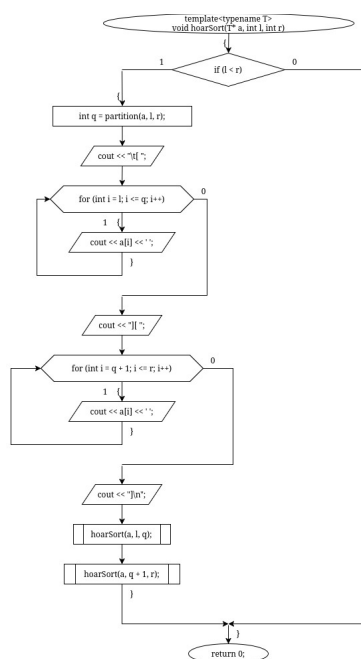
Реализовать алгоритмы сортировок:

1. Сортировка Хоара.
2. Сортировка Ломута.
3. Сортировка Шелла.

АНАЛИЗ АЛГОРИТМОВ ХОАРА И ЛОМУТО

1. Выбирается опорный элемент – pivot
2. Все элементы меньше опорного располагаем слева от него, все что больше справа Шаг
3. Рекурсивно повторяем эти действия, пока не получим отсортированный массив.
4. Отличие этих двух алгоритмов состоит только в выборе опорного элемента и способа размещения элементов относительно него.

БЛОК-СХЕМА ДЛЯ АЛГОРИТМА ХОАРА



КОД

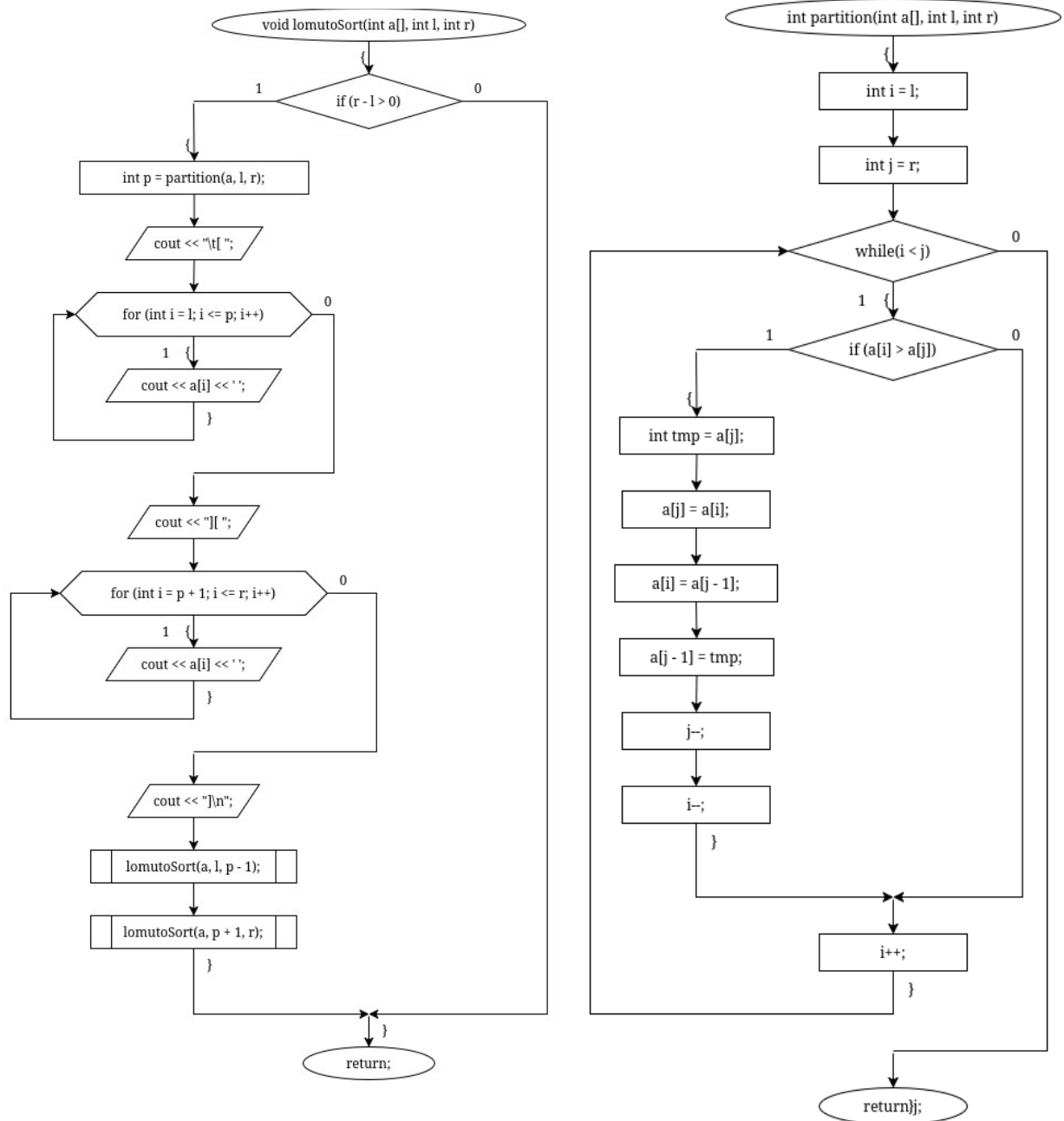
```
#include <iostream>
using namespace std;

template<class T>
int partition(T* a, int l, int r) {
    T v = a[(l + r) / 2];
    int i = l;
    int j = r;
    bool f = true;
    while (i <= j && f) {
        while (a[i] < v) i++;
        while (a[j] > v) j--;
        if (i >= j) {
            f = false;
            continue;
        }

        T tmp = a[i];
        a[i] = a[j];
        a[j] = tmp;
        i++;
        j--;
    }
    return j;
}

template<class T>
void hoarSort(T* a, int l, int r) {
    if (l < r) {
        int q = partition(a, l, r);
        cout << "\t[ ";
        for (int i = l; i <= q; i++) cout << a[i] << ' ';
        cout << "] [ ";
        for (int i = q + 1; i <= r; i++) cout << a[i] << ' ';
        cout << "]\n";
        hoarSort(a, l, q);
        hoarSort(a, q + 1, r);
    }
}
```

БЛОК-СХЕМА АЛГОРИТМА ЛОМУТО



КОД

```
#include <iostream>

using namespace std;

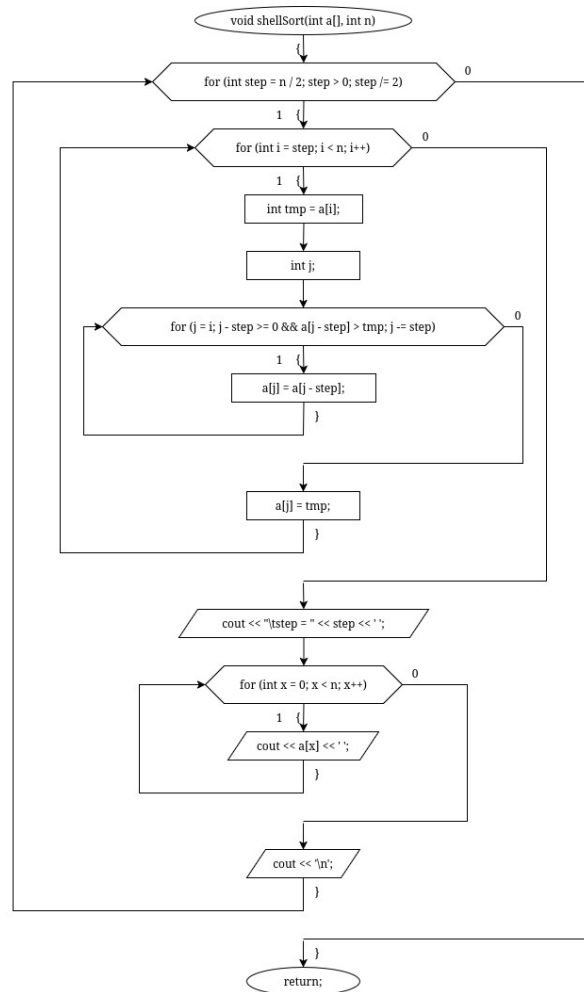
int partition(int a[], int l, int r) {
    int i = l;
    int j = r;
    while (i < j) {
        if (a[i] > a[j]) {
            int tmp = a[j];
            a[j] = a[i];
            a[i] = a[j - 1];
            a[j - 1] = tmp;
            j--;
            i++;
        }
    }
    return j;
}

void lomutoSort(int a[], int l, int r) {
    if (r - l > 0) {
        int p = partition(a, l, r);
        cout << "\t[ ";
        for (int i = l; i <= p; i++) cout << a[i] << ' ';
        cout << "]" << " ";
        for (int i = p + 1; i <= r; i++) cout << a[i] << ' ';
        cout << "]\n";
        lomutoSort(a, l, p - 1);
        lomutoSort(a, p + 1, r);
    }
}
```

АНАЛИЗ АЛГОРИТМА ШЕЛЛА

1. Вычисляется расстояние между элементами – step. Изначально $n / 2$
2. Массив разбивается на списки элементов, отстающих друг от друга на step
3. Элементы каждого списка сортируются методом вставки
4. Происходит объединение списков обратно в массив, step уменьшается в 2 раза, если $step > 0$, то происходит переход обратно на шаг 2.

БЛОК-СХЕМА



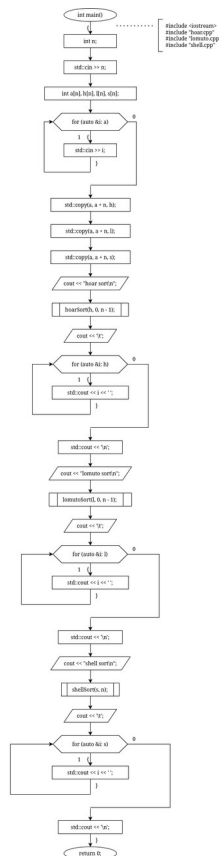
КОД

```
#include <iostream>

using namespace std;

void shellSort(int a[], int n) {
    for (int step = n / 2; step > 0; step /= 2) {
        for (int i = step; i < n; i++) {
            int tmp = a[i];
            int j;
            for (j = i; j - step >= 0 && a[j - step] > tmp; j -= step)
                a[j] = a[j - step];
            a[j] = tmp;
        }
        cout << "\tstep = " << step << ' ';
        for (int x = 0; x < n; x++) cout << a[x] << ' ';
        cout << '\n';
    }
}
```

БЛОК-СХЕМА ФУНКЦИИ int main()



КОД

```
#include <iostream>
#include "hoar.cpp"
#include "lomuto.cpp"
#include "shell.cpp"

int main() {
    int n;
    std::cin >> n;
    int a[n], h[n], l[n], s[n];
    for (auto &i: a) std::cin >> i;
    std::copy(a, a + n, h);
    std::copy(a, a + n, l);
    std::copy(a, a + n, s);

    cout << "hoar sort\n";
    hoarSort(h, 0, n - 1);
    cout << '\t';
    for (auto &i: h) std::cout << i << ' ';
    std::cout << '\n';

    cout << "lomuto sort\n";
    lomutoSort(l, 0, n - 1);
    cout << '\t';
    for (auto &i: l) std::cout << i << ' ';
    std::cout << '\n';

    cout << "shell sort\n";
    shellSort(s, n);
    cout << '\t';
    for (auto &i: s) std::cout << i << ' ';
    std::cout << '\n';
}
```

РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ

7

15 10 8 3 0 -2 -5

hoar sort

[-5 -2 0 3][8 10 15]

[-5 -2][0 3]

[-5][-2]

[0][3]

[8 10][15]

[8][10]

-5 -2 0 3 8 10 15

lomuto sort

[-5][10 8 3 0 -2 15]

[10 8 3 0 -2 15][]

[-2][8 3 0 10]

[8 3 0 10][]

[0][3 8]

[3 8][]

-5 -2 0 3 8 10 15

shell sort

step = 3 -5 0 -2 3 10 8 15

step = 1 -5 -2 0 3 8 10 15

-5 -2 0 3 8 10 15